SWC3

Software-Entwicklung mit C und C++

Erik Pitzer

WS 2026/27

FH OÖ - Campus Hagenberg - Software Engineering



2. C Einführung

»C is quirky, flawed,
and an enormous success.«

— Dennis Ritchie. "Inventing Unix",
Interview with Samuel K. Moore, Steven Cherry,
spectrum.ieee.org. May 27, 2011.

»C is a razor-sharp tool,
with which one can create
an elegant and efficient program
or a bloody mess.«

— Brian W. Kernighan & Rob Pike.
"The Practice of Programming",
Addison-Wesley, 1999

Geschichte von C

- ab 1971 “*new B*” bei Bell Labs von Dennis Ritchie (Turing Award 1983)
  - inspiriert durch
    - *B* (typenlose Sprache) von Ken Thompson (Turing Award 1983)
    - *BCPL* von Martin Richards (IEEE Computer Pioneer Award 2003)
- 1978 “The C Programming Language” von Brian Kernighan und D. Ritchie
  - de-facto Standard *“K&R C”* (C78)
- 1989 ANSI C Standard (C89)
- 1990 ISO C Standard (C90)
- 1995 C95 <wchar.h> <wctype.h>

The development of the C language

- 1999 C99
  - Zeilenendkommentare (`// ...`)
  - Felder mit variabler Länge (VLA)
  - neue Datentypen, z.B. _Complex
  - Fixed width integers z.B. `uint32_t`
- 2011 C11
  - alignment
  - generics
  - unicode
- 2018 C17
  - kleinere Änderungen
  - keine neuen Features
- 2023 C23 moderne Features
  - `constexpr`
  - `typeof`
  - `nullptr`
  - Bitfunktionen <stdbit.h>

Eigenschaften von C (i)

- imperativ vs. deklarativ
  - direkte Kontrolle über Programmablauf und Änderungen
- prozedural vs. objektorientiert oder funktional
  - Organisation in Prozeduren (und Funktionen)
- statisch typisiert vs. dynamisch typisiert
  - Typ einer Variablen ist dem Compiler bekannt
- relativ schwach typisiert vs. stark typisiert
  - Typ einer Variablen kann (relativ leicht) verändert bzw. neu interpretiert werden
- übersetzt vs. interpretiert
  - C wird von einem Compiler in Maschinensprache übersetzt
  - keine Laufzeitumgebung (*runtime*) oder virtuelle Maschine

Eigenschaften von C (ii)

- (relativ) low-level
  - direktere Interaktion mit Hardware und Betriebssystem
  - primitivere Abstraktionskonzepte
- manuelle Speicherverwaltung vs. Garbage Collection
  - planbare Performance
  - weniger Komfort
- portabel und standardisiert vs. plattformspezifisch
  - leichte Verfügbarkeit auf verschiedenen Systemen
- kompakt vs. umfangreich
  - kleine Standardbibliothek
  - hardware-nahe Programmierung (OS, embedded)
- Präprozessor
  - Quelltext wird vor der Übersetzung vorverarbeitet
  - erlaubt Textersetzungen (Makros)

Hauptunterschiede zu Java

- C ist rein prozedural (keine Klassen, keine Objekte)
  - nur strukturierte Datentypen ohne Methoden
- C erlaubt Zeiger und Zeiger-Arithmetik
- C hat eine sehr kleine Standardbibliothek im Vergleich zu Java
- in C werden Funktionsparameter immer per Wert übergeben (call-by-value)
  - Übergabe per Referenz (*call-by-reference*) muss mit Zeiger emuliert werden
  - in Java wird auch per Wert übergeben, Variablen für Objekte sind aber Referenzen und verhalten sich daher eher wie call-by-reference
- in C muss dynamischer Speicher manuell verwaltet werden
  - in Java gibt es einen Garbage Collector
- C Quelltext wird in Header (`*.h`) und Implementierung (`*.c`) getrennt
  - Name der Datei und Pfad sind irrelevant
- C wird zu Maschinensprache übersetzt und direkt vom Prozessor ausgeführt
- C-Zeichenketten sind sehr primitiv

Popularität von C

Quelle	Platz	Stand
TIOBE	2	Sep 2026
PyPL	3	Sep 2026
RedMonk	10	Jun 2025
IEEE Spectrum	8[1]	Sep 2025
GitHut 2.0	8[1]	Mar 2024

[1] Durchschnitt aus mehreren Reihungen

Literatur

Hauptquellen

- J. Gustedt. *“Modern C”*, Manning, 2025
- O. Gazi. *“Modern C Programming”*, Springer, 2024
- C reference - cppreference.com (DevDocs — C documentation)

- M. Kalin. “Modern C Up and Running”, Apress, 2022
- ISO/IEC 9899:2024 *“ISO Standard: C23”* (n3220)
- M. Olsson. *“Modern C Quick Syntax Reference”*, Apress, 2019
- S. Dmitrović. *“Modern C for Absolute Beginners”*, Apress, 2021

- B. Kernighan & D. Ritchie. *“The C programming language”*, Prentice-Hall, 1988 (de)
- Z. Shaw. *“Learn C the hard way”*, Addison-Wesley, 2015

Hello World

- C Quelltext

```c
// file: hello.c

#include <stdio.h>

int main() {
  printf("Hello, World!");
}
```

- Aufruf des Übersetzers
  - z.B. GNU Compiler Collection

```
gcc -o hello hello.c
```

- Aufruf des Programms

```
./hello
```

- Ausgabe des Programms

```
Hello, World!
```

Hello World v2 (i)

```c
// from J. Gustedt "Modern C" (2024)

#include <stdlib.h>
#include <stdio.h>

int main(int argc, [[maybe_unused]] char* argv[argc+1]) {

  double A[5] = {
    [0] = 9.0,
    [1] = 2.9,
    [4] = 3.E+25,
    [3] = 0.0007,
  };

  for (size_t i = 0; i < 5; ++i) {
    printf("A[%zu] = %6g, A[i]² = %8g\n", i, A[i], A[i]*A[i]);
  }

  return EXIT_SUCCESS;
}
```

```
A[0] =      9, A[i]² =       81
A[1] =    2.9, A[i]² =     8.41
A[2] =      0, A[i]² =        0
A[3] = 0.0007, A[i]² =  4.9e-07
A[4] =  3e+25, A[i]² =    9e+50
```

Hello World v2 (ii)

- Kommentare[1] in C entweder `/* inline */` oder als `// Zeilenendkommentar`

```
1  // from J. Gustedt "Modern C" (2024)
```

- Inhalt anderer Dateien inkludieren[2]

```
3  #include <stdlib.h>
4  #include <stdio.h>
```

- Hauptfunktion `main()`, mit Argumenten vom Betriebssystem[3]
  - Anzahl `int argc` (*argument count*) und
  - Inhalt `char* argv[argc+1]` (*argument vector*)
  - Attribut `[[maybe_unused]]`: Parameter muss nicht benutzt werden (seit C23)

```
6  int main(int argc, [[maybe_unused]] char* argv[argc+1]) { /* ... */ }
```

[1] Comments - cppreference.com
[2] Source file inclusion - cppreference.com
[3] Main function - cppreference.com

Hello World v2 (iii)

- Deklaration und Definition eines Feldes (Array)[1] von Fließkommazahlen[2] mit dem Namen `A`
  - direkte Initialisierung an verschiedenen Indizes
  - automatische Initialisierung mit `0` für fehlende Indizes

```
8  double A[5] = {
9    [0] = 9.0,
10   [1] = 2.9,
11   [4] = 3.E+25,
12   [3] = 0.0007,
13 };
```

- Schleife[3] über die Zahlen i $\leftarrow [0..5)$

```
15 for (size_t i = 0; i < 5; ++i) { /* ... */ }
```

[1] Array declaration - cppreference.com
[2] Arithmetic types - Real floating types - cppreference.com
[3] for loop - cppreference.com

Hello World v2 (iv)

- formatierte Ausgabe mit Formatvorlage[1]
  - spezielle Formatangaben (wie `"%zu"` und `"%6g"`) für restlichen Argumente

```
16  printf("A[%zu] = %6g, A[i]² = %8g\n", i, A[i], A[i]*A[i]);
```

- Rückgabewert an das Betriebssystem (`0` → kein Fehler)[2]
  - darf auch weggelassen werden seit C99

```
19  return EXIT_SUCCESS;
```

[1] printf et al. - cppreference.cpp
[2] Main function - cppreference.com

Schlüsselwörter (C23)

Kontrollfluss

- Verzweigung `if` `else` `switch` `case` `default`
- Schleifen `do` `while` `for` `continue` `break`
- Sprünge `return` `goto`

Datentypen

- `char` `unsigned` `signed` `int` `long` `short`
- `void` `float` `double`
- seit C99 `_Complex` `_Imaginary`
- seit C23 `bool` (`!`)
  - `_BitInt` `_Decimal128` `_Decimal32` `_Decimal64`

C keywords - cppreference.com

Typkonstruktoren & -information

- `union` `struct` `enum` `typedef` `sizeof`
- seit C11 `_Atomic` `_Generic`

Modifier

- `auto` `extern` `register` `static` `const` `volatile`
- seit C99 `inline` `restrict`
- seit C23 `constexpr` `alignas` `thread_local`

weitere (seit C23)

- `static_assert` `alignof` `typeof` `typeof_unequal`
- `nullptr` `true` `false` (`!`)

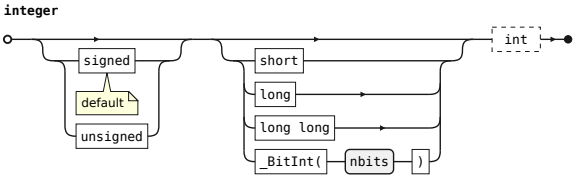
Datentypen Überblick

- Primitive Datentypen
  - Wahrheitswerte `bool` (seit C23)
    - mit Werten `true` oder `false`
    - früher `_Bool` mit `1` (= wahr) und `0` (= falsch)
  - Zeichen `char` in Zeichenketten
    - `char8_t` für UTF-8 Zeichen
  - Ganze Zahlen (*integer* mit Vorzeichen)
    - `signed char`, `short`, `int`, `long`, `long long`
  - Natürliche Zahlen
    - `unsigned char`, `unsigned short`, `unsigned int`, `unsigned long`, `unsigned long long`
  - Fließkommazahlen
    - `float`, `double`, `long double`
    - Dezimale Fließkommazahlen z.B. `_Decimal32`
    - Komplexe und imaginäre Zahlen, z.B. `float _Complex` oder `float _Imaginary`
- *“kein Wert”* `void` (engl. *leer*)
- Aufzählungstypen `enum`
- Abgeleitete Datentypen
  - Felder (*arrays*), z.B. `int a[]`
  - Verbunde (*records*) `struct`
  - Vereinigungstypen `union`
  - Funktionstypen, z.B. `int sum(int a, int b)`
  - Zeiger (*pointer*), z.B. `int *ptr`
  - Atomare Typen (für Nebenläufigkeit), z.B. `_Atomic int ai`

Type System Overview - cppreference.com

Ganze Zahlen

- Ganzzahlige Datentypen (engl. *integer*) werden angegeben durch
  - Vorzeichen (engl. *signedness*): `signed` (default) oder `unsigned`
  - Länge: `short`, keine Angabe (default), `long`, `long long`, oder `_BitInt()`
  - `int` (optional falls Modifier verwendet)



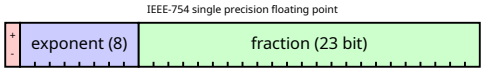
- Der Wertebereich ist compilerabhängig
  - Standard schreibt nur relative Größe vor
  - `1 == sizeof(char) ≤ sizeof(short) ≤ sizeof(int) ≤ sizeof(long) ≤ sizeof(long long)`
- Bei vorzeichenlosen Zahlen (`unsigned`) *wohldefinierter* overflow z.B. `UINT_MAX+1` → `0`
- Bei ganzen Zahlen (`signed`) undefined behavior

Arithmetic Types - cppreference.com

Fließkommazahlen & Complexe Zahlen

- implementieren den Standard IEEE 754
- binäre Fließkommazahlen
  - `float` (32bit)
  - `double` (64bit)
  - `long double` (128bit)
- binär kodierte Dezimalzahlen (BCD)
  - `_Decimal32`
  - `_Decimal64`
  - `_Decimal128`
- Komplexe Zahlen (bestehend aus Real- und Imaginärteil)
  - `float _Complex`
  - `double _Complex`
  - `long double _Complex`

Arithmetic types - Real floating types - cppreference.com




```c
double dbl     = 0.1;   // 64bit
_Decimal32 d32 = 0.1df; // 32bit

SHOW_DBL(dbl);
SHOW_D32(d32);
```

```
    double: 0.1000000000000000055511151
_Decimal32: 0.1000000000000000000000000
```

```c
double _Complex c = -1.0;
SHOW_COMPLEX(c);
SHOW_COMPLEX(sqrt(c));
```

```
c = -1+0i
sqrt(c) = 0+1i
```

Keine fixen Wertebereiche für Zahlen

- Wertebereiche für Zahlen-Datentypen sind nur relativ zueinander standardisiert
- können von Compiler und Zielplattform oder -architektur abhängen
- Grenzen für ganze Zahlen sind im Header <limits.h> definiert
- Bereiche für Fließkommazahlen finden sich im Header <float.h>

```c
#include <limits.h>
#include <float.h>

int main() {
  SHOW2(CHAR_MIN, CHAR_MAX);
  SHOW(UINT_MAX);
  SHOW2(FLT_MIN, FLT_MAX);
}
```

```
CHAR_MIN = -128;  CHAR_MAX = 127
UINT_MAX = 4294967295
FLT_MIN = 1.17549e-38;  FLT_MAX = 3.40282e+38
```

Wertebereiche *Beispiel* — für GCC und x86_64 unter Linux

Typ	Bits	Minimum	Maximum
`char`	8	-128	127
`unsigned char`	8	0	255
`signed char`	8	-128	127
`short`	16	-32.768	32.767
`signed short`	16	-32.768	32.767
`unsigned short`	16	0	65.535
`int`	32	-2.147.483.648	2.147.483.647
`signed`	32	-2.147.483.648	2.147.483.647
`unsigned`	32	0	4.294.967.295
`long`	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
`signed long`	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
`unsigned long`	64	0	18.446.744.073.709.551.615
`long long`	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
`signed long long`	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
`unsigned long long`	64	0	18.446.744.073.709.551.615
`float`	32	1,17549e-38	3,40282e+38
`double`	64	2,22507e-308	1,79769e+308
`long double`	128	3,3621e-4932	1,18973e+4932

Datentypen mit fixer Größe

- Datentypen mit fixer Größe werden in <stdint.h> definiert

Typ	Bits	Minimum	Maximum
`uint8_t`	8	0	255
`uint32_t`	32	0	4.294.967.295
`uint64_t`	64	0	18.446.744.073.709.551.615
`int8_t`	8	-128	127
`int32_t`	32	2.147.483.648	2.147.483.647
`int64_t`	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
`int_least8_t`	8	-128	127
`int_least32_t`	32	2.147.483.648	2.147.483.647
`int_least64_t`	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
`int_fast8_t`	8	-128	127
`int_fast32_t`	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
`int_fast64_t`	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
`intmax_t`	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
`uintmax_t`	64	0	18.446.744.073.709.551.615

Literale

- Literale sind Elemente der Sprache die *buchstäblich* für sich selbst stehen

- *nackte* Zahlenwerte Werte
  - `true` und `false` → `bool`
  - `5` oder `-3` → `int`
  - `5.5` oder `0.123e-12` → `double`
- Zahlen mit Basis-Präfix
  - `0x` hexadezimal
    - `0xff` → 255
    - `0x1p-1` → 0.5
    - `0x1p2` → 4.0
  - `0` oktal (!): `020` → 16
  - `0b` binär: `0b101` → 5
- Zahlen mit Typ-Suffix
  - `0u` → `unsigned int`
  - `0l` → `long`
  - `0ll` → `long long`
  - `0f` → `float`
  - `0ld` → `long double`

Boolean literals - cppreference.com
Integer Literals - cppreference.com
Floating-point literals - cppreference.com

- Kombinationen von Präfix und Suffix
  - `0b1010u` → `(unsigned)10`
- Zifferntrenner für Zahlen
  - `1'250'399` → $1\,250\,399$
- einzelne Zeichen (*character*)
  - `'a'` vom Typ `int` (!)
  - `u8'a'` vom Typ `char8_t`
    - UTF-8 codiertes Zeichen
- Zeichenkettenkonstanten
  - `"Hello World!\n"` → `const char[]`
  - `u8"I♡C ☺"` → `const char8_t[]`)
  - keine Längeninformation
  - Terminatorzeichen (ASCII Code `0`)
- Escape-Codes für Zeichen
  - z.B. `'\n'` für Zeilenumbruch, `'\0'` für Terminatorzeichen

Character literals - cppreference.com

Typumwandlung (*type cast*) (i)

- Datentypen können mittels Typecast umgewandelt werden
  - dabei können
    - Informationen verloren gehen
    - Umwandlungsfehler entstehen
  - in der Praxis daher sehr sparsam einsetzen

```
SHOW_TYPE('c');
SHOW_TYPE((char)'c');
```

```
'c' = 99 (int)
(char)'c' = 'c' (char)
```

```
SHOW_TYPE(INT_MAX);
SHOW_TYPE((short)INT_MAX);
```

```
INT_MAX = 2147483647 (int)
(short)INT_MAX = -1 (short int)
```

```
SHOW_TYPE(0.1);
SHOW_TYPE((int)7.9);
SHOW_TYPE(DBL_MAX);
SHOW_TYPE((float)DBL_MAX);
```

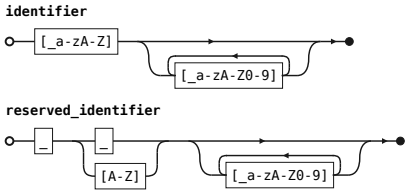
```
0.1 = 0.1 (double)
(int)7.9 = 7 (int)
DBL_MAX = 1.79769e+308 (double)
(float)DBL_MAX = inf (float)
```

Bezeichner (*identifier*)

- Bezeichner (engl. *identifier*) sind alle Namen die nicht von der Sprache vordefiniert sind und werden vergeben für
  - Variablen, z.B. `int counter`
  - Konstanten, z.B. `#define MAX_LENGTH = 12`
  - Datentypen, z.B. `struct time_t`
  - Funktionen, z.B. `printf()`
- jeder Name kann pro Scope 1x vergeben werden, z.B. für Funktion oder Variable

- reservierte Bezeichner
  - beginnend mit zwei Unterstrichen
    - z.B. `__STDC_VERSION__`
  - Beginnend mit einem Unterstrich und einem Großbuchstaben
    - z.B. `_Bool`
- lokale Bezeichner
  - beginnend mit einem Unterstrich gelten nur innerhalb einer Datei
    - z.B. `_local`

- Typischerweise definiert durch folgenden regulären Ausdruck



Deklaration und Definition

- Vor Verwendung eines Bezeichners muss dieser deklariert werden
  - die Deklaration gibt an wofür ein Bezeichner steht
  - kann mehrmals deklariert werden (solange sich Deklarationen nicht widersprechen)

```c
int argc;
size_t count;
double A[5];
int main(int, char*[]);
```

  - jede Deklaration beginnt mit der Angabe des Typs
  - eckige Klammern deklarieren ein Feld (engl. *array*)
  - runde Klammern deklarieren eine Funktion
  - ein Asterisk (`*`) deklariert einen Zeiger (engl. *pointer*)
- Die Definition gibt den Wert oder den Inhalt sowie den Speicherort an
- Jeder Wert kann nur einmal (pro Scope) definiert werden

Deklaration und Definition – Beispiel: Variable

- Deklaration besagt

```
size_t count;
```

  - *irgendwo* gibt es genug Speicher
  - für einen Wert vom Typ `size_t`
  - unter dem Namen `count`
- Hierbei gilt es folgende Teile zu unterscheiden
  - Name des Objekts (`count`) ist ein Bezeichner
  - Spezifikation (der Typ) ist `size_t`
  - Objekt selbst, z.B. Stelle im Speicher für die Variable
  - Wert des Objekts, z.B. `0`

- Definition besagt, dass im aktuellen Scope

```
size_t count = 0;
// alternativ auch
size_t count = {};
```

  - unter dem Namen `count`
  - genug Platz für Wert vom Typ `size_t` reserviert wird
  - und mit `0` initialisiert wird
- korrekte Null-Initialisierung durch `... = {};`
- C Idiom »Zero is initialized« (ZII)
  - Datenstrukturen so konzipieren, dass `0` eine gültige, z.B. leere Initialisierung ist
  - bedeutet aber nicht, dass alle Variablen automatisch mit `0` initialisiert werden

Deklaration und Definition – Beispiel: Funktion

- Deklaration besagt

```
int count(short);
```

  - *irgendwo* gibt es genug Speicher
  - für eine Funktion vom Typ `int(short)`
    - nimmt einen `short`
    - liefert einen `int`
  - unter dem Namen `count`

- Definition besagt, dass im aktuellen Scope

```
int count(short x) { /* ... */ }
```

  - unter dem Namen `count`
  - genug Platz für eine Funktion
  - vom Typ `int(short)` reserviert wird

- Hierbei gibt es wieder
  - Name des Objekts (`count`) ist ein Bezeichner
  - Spezifikation (der Typ) ist `int(short)`
  - Objekt selbst, z.B. Stelle im Speicher die Funktion
  - Wert (Implementierung) des Objekts (in Maschinensprache)

Deklaration und Definition – Programmierstil

- Deklaration und Definition möglichst nahe oder gleichzeitig
  - Reduktion von nicht initialisierten Variablen

```
int x = 1;
```

- Immer nur eine Definition oder Deklaration pro Zeile
  - C erlaubt eine Definition fortzusetzen
  - verliert dabei aber Erweiterungen wie Zeiger oder Feld

```
int*   a, b; // ==> int *a; int b;  (misleading)
int   *a, b; // ==> int *a; int b;  (a bit better)
```

  - am besten separat definieren (und initialisieren)

```
int *a = {};
int b = {};
```

Konstanten (i)

- `const`-*qualifizierte* Variablen
  - können in schreibgeschütztem Speicher liegen
  - keine (direkten) Änderungen
  - Änderungsversuch → undefiniertes Verhalten

```
const int x = 1; // or `int const x = 1;`
SHOW(x);
// x = 2; <-- does not compile
*(int*)(&x) = 2; // XXX: remove const via cast
SHOW(x);
const int r = rand(); // initialize with function
SHOW(r);
```

```
x = 1
x = 2
r = 1804289383
```

const type qualifier - cppreference.com

- `constexpr` Kennzeichnung (seit C23)
  - echte Konstante: kann nicht verändert werden
  - muss zur Übersetzungszeit bekannt sein
  - kann nicht mit Funktion initialisiert werden

```
constexpr int x = 1;
SHOW(x);
// x = 2; <-- does not compile
*(int*)(&x) = 2; // NOTE: cannot change x
SHOW(x);
// constexpr r = rand(); <-- ERROR: does not compile
```

```
x = 1
x = 1
```

constexpr specifier - cppreference.com

Konstanten (ii)

- vor C23 wurden echte Konstanten daher oft mit Hilfe des Präprozessors definiert
  - einfache Textersetzung

```
#define PI 3.1415
SHOW_TYPE(PI);

// equivalent to

SHOW_TYPE(3.1415);
```

```
PI = 3.1415 (double)
3.1415 = 3.1415 (double)
```

Weitere Zusätze

- Variablen können als unbeständig »`volatile`« deklariert werden
  - Compiler muss davon ausgehen, dass sich Variable *unbemerkt* ändern kann
  - Optimierung wird eingeschränkt
  - Wert wird immer direkt aus dem Speicher gelesen
  - z.B. Speicherabbildung (*memory mapping*) von Geräten durch Treiber
  - z.B. Variable die von anderem Thread oder Signal-Handler verwendet wird
- Zeiger (*pointer*) können eingeschränkt werden »`restrict`«
  - Compiler darf annehmen, dass sich diese Zeiger (und alles darin) nicht überlappen
  - ermöglicht zusätzliche Optimierungen

Gültigkeitsbereiche (*scope*) & Lebensdauer (*lifetime*) (i)

Bezeichner haben verschiedene Gültigkeitsbereiche (engl. *scope*)

- global: überall verfügbar
- innerhalb einer Datei (`static`)
- innerhalb eines *Blocks* { `/* ... */` }
  - Funktion
  - Schleife, Verzweigung, ...
  - anonymer Block

innerhalb eines Scopes sind alle Bezeichner

- des gleichen und
- aller umgebenden Scopes sichtbar
- die vorher deklariert wurden

Variablen haben verschiedene Lebensdauer (engl. *lifetime* oder *extent*)

- global (immer verfügbar)
  - außerhalb von Funktionen
  - innerhalb von Funktionen (`static`)
- solange Block ausgeführt wird
  - Funktion
  - Schleife, Verzweigung, ...
  - anonymer Block

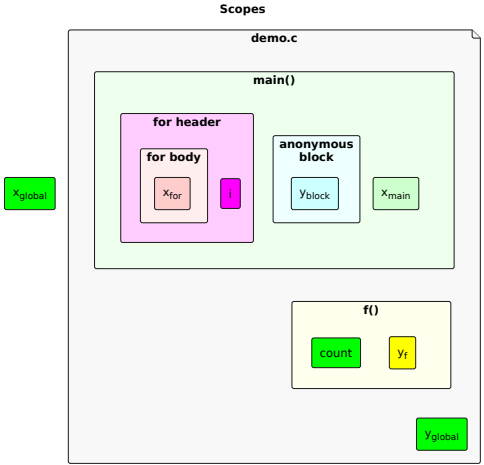
Gültigkeitsbereiche (*scope*) & Lebensdauer (*lifetime*) (ii)

```c
// file: demo.c
int x = {}; // global x_global
static int y = {}; // file-private global y_global

void f(void) {
  int y = {}; // function-local y_f

  // function-private global count
  static int count = {};
}

int main() {
  int x = {}; // function-local x_main
  for (int i = 0; i<3; i++) /* for-header i */ {
    int x = i+1; // for-body x_for
  }
  {
    int y = 1; // anonymous block y_block
  }
}
```



Farbe	Lifetime	Farbe	Lifetime
	jederzeit		während gesamter for-Schleife
	`main()`		während einzelner Iteration
	`f()`		während des Blocks

Gültigkeitsbereiche (*scope*) & Lebensdauer (*lifetime*) (iii)

```c
// file: demo.c
int x = {}; // global x_global
static int y = {}; // file-private global y_global

void f(void) {
  int y = {}; // function-local y_f

  // function-private global
  static int count = {};
}

int main() {
  int x = {}; // function-local x_main
  for (int i = 0; i<3; i++) /* for-header i */ {
    int x = i+1; // for-body x_for
  }
  {
    int y = 1; // anonymous block y_block
  }
}
```

- Variablen auf höchster Ebene also außerhalb von Funktionen sind verfügbar
  - scope: überall
  - lifetime: jederzeit
  - z.B. `2 int x = {}`
- *globale* `static` Variablen sind verfügbar
  - scope: nur innerhalb der Datei
  - lifetime: jederzeit
  - z.B. `3 static int y = {}`

Gültigkeitsbereiche (*scope*) & Lebensdauer (*lifetime*) (iv)

```c
// file: demo.c
int x = {}; // global x_global
static int y = {}; // file-private global

void f() {
  int y = {}; // function-local y_f

  // function-private global
  static int count = {};
}

int main() {
  int x = {}; // function-local x_main
  for (int i = 0; i<3; i++) /* for-header i */ {
    int x = i+1; // for-body x_for
  }
  {
    int y = 1; // anonymous block y_block
  }
}
```

lokale Variablen innerhalb von Blöcken sind verfügbar

- scope:
  - innerhalb des jeweiligen Blocks
  - in allen darin befindlichen Blöcken
- lifetime: solange Block ausgeführt wird
- z.B.
  - `6 int y = {}`
  - `13 int x = {}`
  - `15 int x = i+1`
  - `18 int y = {}`

bei wiederholtem Durchlauf werden Sie neu angelegt (scope+lifetime)

- z.B. erneuter Funktionsaufruf oder Schleifendurchlauf

Gültigkeitsbereiche (*scope*) & Lebensdauer (*lifetime*) (v)

```c
// file: demo.c
int x = {}; // global x_global
static int y = {}; // file-private global y_global

void f() {
  int y = {}; // function-local y_f

  // function-private global
  static int count = {};

}

int main() {
  int x = {}; // function-local x_main
  for (int i = 0; i<3; i++) /* for-header i */ {
    int x = i+1; // for-body x_for
  }
  {
    int y = 1; // anonymous block y_block
  }
}
```

lokale `static` Variablen werden bei weiteren Funktionsaufrufen wieder verwendet

- scope wie lokale Variablen
  - innerhalb des gleichen Blocks
- lifetime wie globale Variablen (!)
- z.B. 9 `static int count = {}`
- think `static` ≈ `private global`

Gültigkeitsbereiche (*scope*) & Lebensdauer (*lifetime*) (vi)

```c
// file: demo.c
int x = {}; // global x_global
static int y = {}; // file-private global y_global

void f() {
  int y = {}; // function-local y_f

  // function-private global
  static int count = {};
}

int main() {
  int x = {}; // function-local x_main
  for (int i = 0; i<3; i++) /* for-header i */ {
    int x = i+1; // for-body x_for
  }
  {
    int y = 1; // anonymous block y_block
  }
}
```

- Schleifen-Kopf-Variablen sind verfügbar
  - scope
    - im Schleifenkopf und
    - im Schleifenrumpf
  - lifetime: existieren während aller Schleifendurchläufe
  - z.B. `14 int i = 0`
- Schleifen-Rumpf-Variablen sind verfügbar
  - scope: nur im Schleifen-Rumpf
  - lifetime: nur für eine Iteration gültig, werden also bei jedem Durchlauf neu angelegt
  - `15 int x = i+1`

Gültigkeitsbereiche (*scope*) & Lebensdauer (*lifetime*) (vii)

```c
// file: demo.c
int x = {}; // global x_global
static int y = {}; // file-private global y_global

void f() {
  int y = {}; // function-local y_f

  // function-private global
  static int count = {};
}

int main() {
  int x = {}; // function-local x_main
  for (int i = 0; i<3; i++) /* for-header i */ {
    int x = i+1; // for-body x_for
  }
  {
    int y = 1; // anonymous block y_block
  }
}
```

- Wenn Bezeichner wieder verwendet werden, überschatten sie die weiter außen liegenden Bezeichner
  - `2 int x = {}` wird in
  - `main()` durch `13 int x = {}` überschattet
  - welches selbst wiederum von `15 int x = i+1` innerhalb der `for`-Schleife überschattet wird

Gültigkeitsbereiche (*scope*) & Lebensdauer (*lifetime*) (viii)

```c
// file: demo.c
int x = {}; // global x_global
static int y = {}; // file-private global y_global

void f(void) {
  int y = {}; // function-local y_f

  // function-private global
  static int count = {};
}

int main() {
  int x = {}; // function-local x_main
  for (int i = 0; i<3; i++) /* for-header i */ {
    int x = i+1; // for-body x_for
  }
  {
    int y = 1; // anonymous block y_block
  }
}
```

- Wenn Bezeichner wieder verwendet werden, überschatten sie die weiter außen liegenden Bezeichner
  - 3 `static int y = {}`
    - wird in `f()` von 6 `int y = {}` überschattet
    - ist in `main()` verfügbar
    - wird im anonymen Block von 18 `int y = 1` überschattet

Speicherklassen (*storage classes*)

- `auto` automatische Lifetime, außen nicht sichtbar
  - default für lokale Variablen
  - `register` ermutigt den Compiler CPU-Register zu verwenden
- `static` innerhalb des aktuellen Blocks (scope) dauerhaft (lifetime)
  - auch bei erneutem Durchlauf
- `extern` global sichtbar (scope) und dauerhaft (lifetime)
  - default für globale Variablen
  - explizit normalerweise nur für Deklaration → Variable/Funktion außerhalb

```c
void function();
extern void function(); // equivalent

int x;
extern int x; // equivalent

int main() {
  int x = {};
  auto int x = {} // equivalent
  register int y = {};
}
```

```c
// file: extern-1.c

extern int x; // -->
int print_x() {
  SHOW(x);
}
```

```c
// file: extern-2.c
int x = 1;
extern int print_x(); // -->
int main() {
  print_x();
}
```

```
gcc -o extern extern-1.c extern-2.c
./extern
```

```
x = 1
```